

Who Wrote It?

Identifying LLMs from Their Responses

CMPSC 448 Midterm Project Report

Aidan Eckfeldt

Individual Project

Penn State University, Fall 2026

Abstract and Research Questions

This project tests whether neural text classifiers can identify which large language model generated a response. A balanced 900-example dataset was curated from the public LMArena arena-human-preference-100k dataset, using GPT-4o, Claude 3.5 Sonnet, and Gemini 1.5 Pro across general, code, and math domains. A convolutional neural network (CNN) and a bidirectional long short-term memory network (LSTM) were trained and evaluated.

For output-only classification, the CNN reached 78.5% accuracy and the LSTM reached 74.1%, well above the 33.3% three-class chance level. Input-only classification stayed near chance. Adding the prompt did not improve over output-only classification. Cross-domain tests remained substantially above chance, and a formatting ablation reduced CNN accuracy from 78.5% to 65.9%, indicating that structural formatting is part of the detectable model fingerprint.

Research questions

- RQ1: Can the model that generated a response be identified from the LLM output alone?
- RQ2: Does the user prompt help identify the LLM? Compare input-only, output-only, and input + output.
- RQ3: Do model fingerprints generalize across task domains?
- RQ4: What response characteristics distinguish the models, and how much does formatting contribute?

1. Dataset Curation

Source. The dataset was curated from LMArena arena-human-preference-100k, a public release of Chatbot Arena conversation data. The source contains user prompts, model identities, model responses, language, code indicators, refusal indicators, and category metadata.

Model selection. One exact model version was selected from each family to avoid mixing multiple versions inside a class: GPT-4o-2024-05-13, Claude-3-5-Sonnet-20240620, and Gemini-1.5-Pro-API-0514.

Filtering and balancing. Only English, single-turn, non-refusal conversations were retained. Three domains were used: general, code, and math. For each model family and domain, 100 examples were sampled, producing 900 total examples.

Split strategy. Train, validation, and test partitions were created with GroupShuffleSplit using question_id as the grouping variable. The resulting split contained 631 training rows, 134 validation rows, and 135 test rows. Grouping prevents the same underlying prompt identifier from appearing in more than one split and reduces prompt leakage.

Family	Exact model	General	Code	Math	Total
GPT	gpt-4o-2024-05-13	100	100	100	300
Claude	claude-3-5-sonnet-2024	100	100	100	300
Gemini	gemini-1.5-pro-api-05	100	100	100	300

2. CNN/RNN Implementation and Training

Both models were implemented in PyTorch. Text was lowercased and tokenized with a regular expression that separates words and punctuation. The vocabulary was built only from the training split. Unknown and padding tokens were handled explicitly.

CNN. The CNN used a 128-dimensional embedding layer followed by three one-dimensional convolution branches with kernel sizes 3, 4, and 5. Each branch used 128 filters. ReLU activation and global max pooling produced fixed-size feature vectors, which were concatenated, passed through dropout of 0.3, and classified with a linear layer.

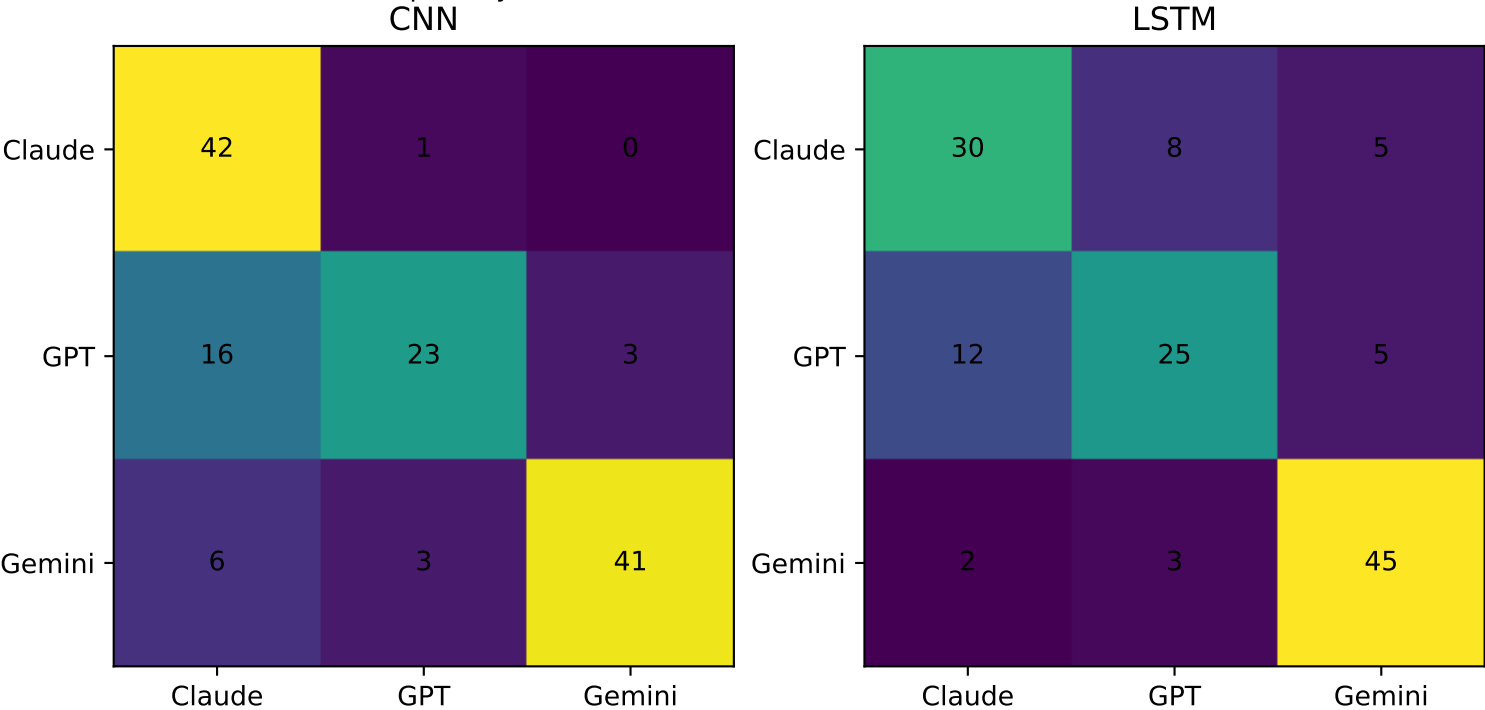
LSTM. The recurrent model used a 128-dimensional embedding layer and a bidirectional LSTM with hidden size 128. The final forward and backward hidden states were concatenated, passed through dropout of 0.3, and classified with a linear layer.

Training. Both models used cross-entropy loss and Adam with learning rate 0.001. Training ran for up to 8 epochs with early stopping after two validation epochs without improvement. Gradient norm clipping at 5.0 was used. The main experiments used batch size 64.

Sequence lengths. Input-only and output-only experiments used a maximum sequence length of 256 tokens. An initial input + output run used the same single 256-token budget, but post-run inspection showed that this frequently truncated model output. The corrected combined condition therefore reserved 128 prompt tokens and 384 output tokens separated by a special token. The corrected combined results are used in this report.

3. RQ1: Output-Only Classification

Both required architectures classified output-only text far above the 33.3% chance level. The CNN reached 78.5% accuracy with macro F1 77.5%. The LSTM reached 74.1% accuracy with macro F1 72.9%. Gemini was especially identifiable; GPT was the hardest class for the CNN.



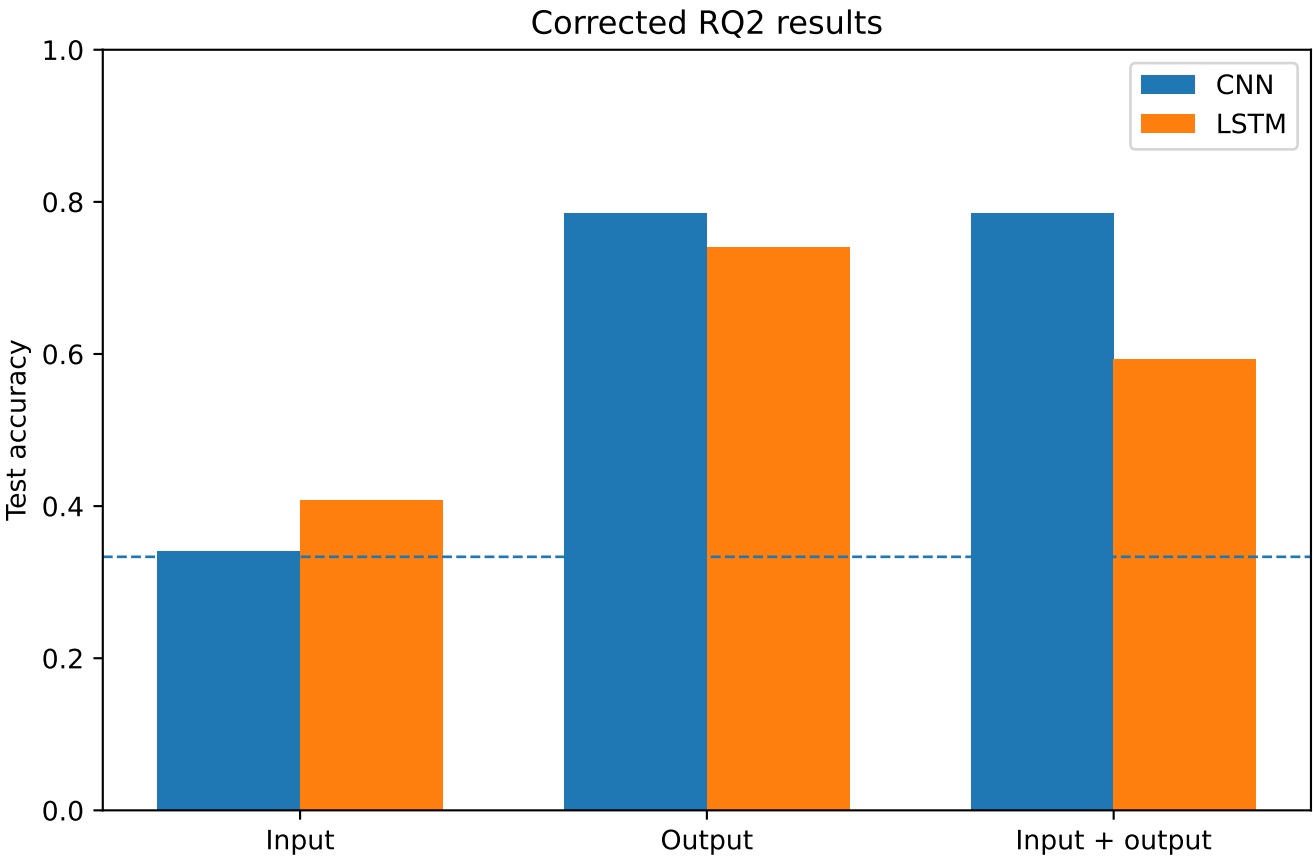
Output-only performance

CNN: accuracy 78.5%, macro F1 77.5%

LSTM: accuracy 74.1%, macro F1 72.9%

4. RQ2: Does the User Prompt Help?

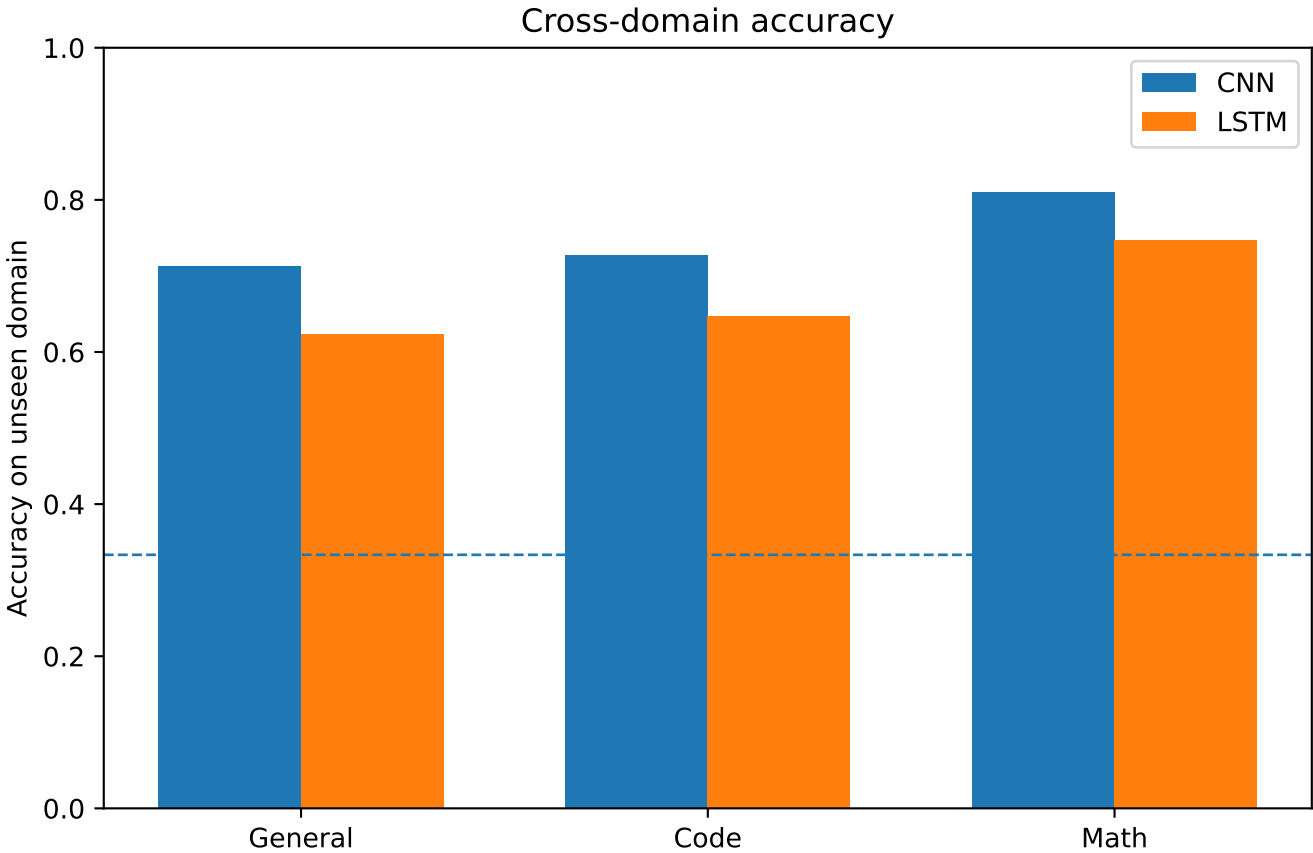
The prompt by itself performed near chance. Output-only text was substantially more informative. After correcting the combined-sequence truncation issue, input + output matched output-only accuracy for the CNN and remained below output-only for the LSTM. This suggests that most model-identifying information lies in the generated response.



The input-only results also act as a sanity check: there is little evidence that model identity can be inferred from the human prompt alone.

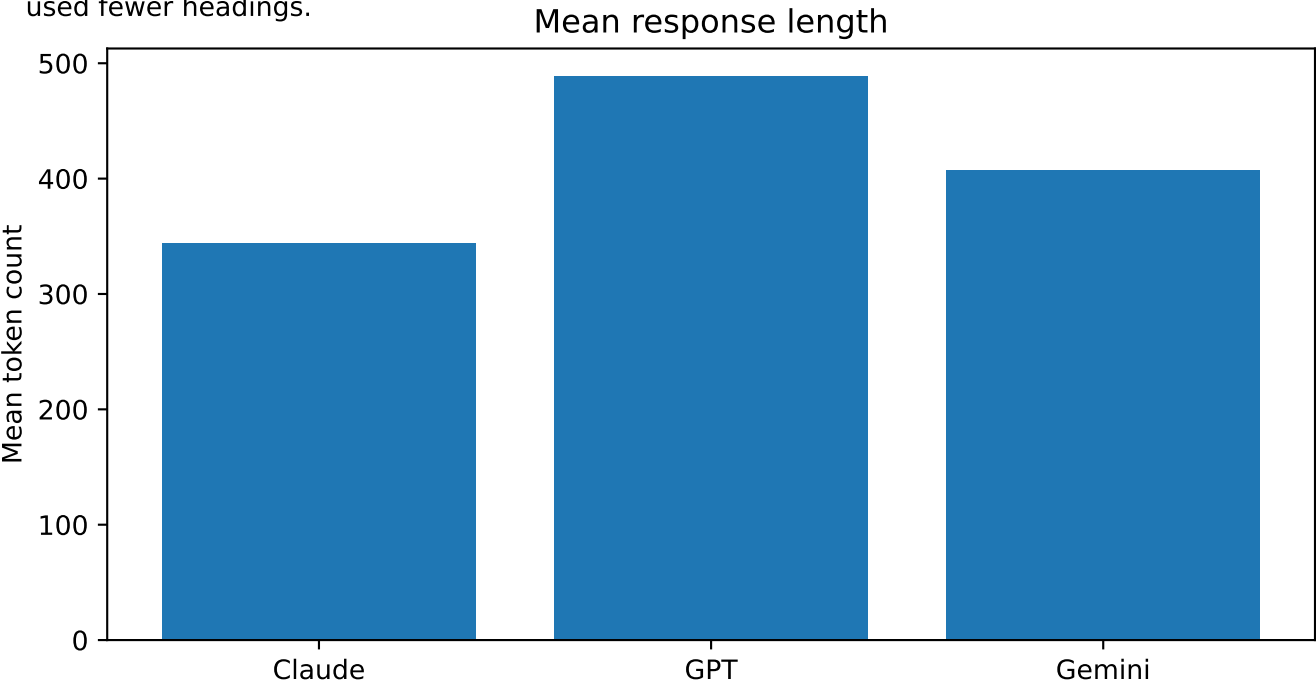
5. RQ3: Cross-Domain Generalization

One domain was held out entirely. Each classifier was trained and validated on the other two domains, then evaluated on the unseen domain using output-only text. Performance remained well above chance for every held-out domain, supporting the interpretation that at least some learned signals generalize across domains.



6. RQ4: Characteristics and Formatting Ablation

Simple structural and lexical characteristics differed across model families. GPT responses were longest on average and used the most headings and code fences. Gemini used the most bullet points and had the highest lexical diversity. Claude responses were shortest on average and used fewer headings.



Formatting ablation

Common Markdown structure was stripped before retraining the output-only CNN. Accuracy fell from 78.5% to 65.9%, a decrease of 12.6 percentage points. Formatting is therefore a meaningful part of the classifier's signal, although it is not the only signal because accuracy remained well above chance after stripping it.

7. Discussion, Limitations, and Lessons

Discussion. Both neural architectures can identify the source LLM from response text at a level far above chance. The CNN performed better than the LSTM on the primary output-only task and on every cross-domain test. A plausible explanation is that local lexical and formatting patterns are strong signals in this dataset, which suits convolutional filters and max pooling.

The experiment also shows why preprocessing matters. The first combined-input implementation placed prompt and output inside one 256-token window. Long prompts frequently consumed much of that window. A targeted rerun reserved separate token budgets for prompt and response, materially improving the combined LSTM result and changing the scientific interpretation.

Limitations. The data comes from a specific Chatbot Arena period and exact model versions, so fingerprints may change for newer versions. The domains are broad labels. The curated examples are balanced by model and domain, but exact prompts are not necessarily identical across all model classes. Formatting and response length may reflect default model behavior as well as deeper linguistic style.

Lessons. Dataset design matters as much as model choice. Input-only accuracy provides a useful sanity check. Preprocessing choices can change the conclusion. Simple models can capture meaningful stylistic signals. Accuracy alone is insufficient, so confusion matrices, cross-domain testing, feature statistics, and an ablation were used to interpret the classifier.

8. Reproducibility and References

The repository contains dataset-construction code, CNN/LSTM experiment code, the targeted corrected RQ2 rerun, result CSVs, figures, and this report. Random seed 448 was used for Python, NumPy, PyTorch, data sampling, and grouped splitting where applicable.

References

- [1] LMArena. arena-human-preference-100k dataset, Hugging Face.
- [2] W.-L. Chiang et al. "Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference." arXiv:2403.04132, 2024.
- [3] PyTorch documentation. `torch.nn.Conv1d`.
- [4] PyTorch documentation. `torch.nn.LSTM`.